



Considerações Práticas sobre Testes de Software

No módulo anterior, foram apresentados alguns conceitos sobre testes de software, como sua motivação e tipos de teste, para compreendermos sua importância para a qualidade do software ao atuar proativamente para identificar e corrigir erros. Nesta aula, iremos entender como realizar estes testes em maiores detalhes, com algumas considerações práticas.

Dentre as estratégias de teste, podemos utilizar testes automatizados ou manuais. Para criação de testes automatizados, é possível utilizar frameworks genéricos de testes e outras ferramentas que facilitam sua criação (SOMMERVILLE, 2019). Os testes manuais podem ser realizados por pessoas, sem o uso de ferramentas automatizadas (desenvolvedores, equipes específicas de teste ou usuários em potencial).

Por exemplo, consideremos um teste unitário, em que uma classe ou parte menor de um programa está sendo testado. Estes testes unitários podem ser automatizados, de forma que podemos usar programas para testar outros programas.

Uma vantagem da automação dos testes é que, sempre que o programa for modificado, é possível realizar os testes automatizados em segundos e ter certeza que algum teste bem-sucedido anteriormente não passe a ser mal sucedido com a nova versão do programa.

Outro aspecto relacionado às estratégias de teste é o uso de mock objects. Quando componentes específicos de software dependem (isto é, se integram) com partes externas, é comum o uso de mocks para simular

interfaces de outros sistemas, inclusive simulando eventos anormais em cenários de teste específicos (SOMMERVILLE, 2019).

As dimensões do teste estão relacionadas aos casos de teste que devem ser especificados, sejam os testes realizados de forma automática ou manual. Ao testar um software, podemos tentar verificar se o código está se comportando conforme o esperado, considerando uma situação de uso convencional. Por outro lado, ao realizar testes também é possível identificar comportamentos fora do comum em alguns fluxos alternativos de uso (SOMMERVILLE, 2019).

Os testes também podem ser realizados em diversos níveis, isto é, assim como podemos modelar um sistema em diversos níveis de abstração, podemos especificar casos de teste em diferentes níveis do sistema. Os testes unitários são um exemplo de um tipo de teste em um nível bem mais baixo, enquanto um teste do sistema completo pode ser considerado um teste em alto nível do sistema. Observe que em ambos os casos temos pré-condições, pós-condições, procedimentos de teste e resultados esperados, mas sua especificação é dependente do nível de teste que se deseja realizar. Por exemplo, os resultados esperados de um teste unitário podem ser valores numéricos que só fazem sentido em um contexto mais amplo, e o resultado esperado do teste do sistema completo pode ser uma mensagem de e-mail enviada com a compilação de resultados financeiros de uma empresa.

Quando o teste é especificado baseado em uma especificação de um sistema, temos um teste do tipo caixa preta. Neste tipo de teste, não é necessário entender os detalhes de funcionamento do sistema, ou seja, como ele consegue suportar os requisitos especificados (SOMMERVILLE, 2019). Por exemplo, podemos ter uma especificação de um sistema de rega automática de plantas, e mesmo sem ter conhecimento das tecnologias de Internet das Coisas utilizadas, podemos realizar o teste de diversos cenários de umidade e temperatura e observar se a rega

automática se comporta conforme especificado. Ao se comprar ou contratar software de terceiros, também pode ser útil realizar testes para verificar que a especificação está de acordo com o que foi entregue. Desta forma, eventuais defeitos de softwares de terceiros podem ter impactos minimizados no software próprio da empresa.

Uma alternativa para se definir testes é observar o código do programa para derivar testes possíveis, o que denominamos de testes caixa branca (SOMMERVILLE, 2019). Por exemplo, se há um tratamento de exceção em um programa, podemos definir um teste relacionado. Por outro lado, podemos observar condicionais em um código para realizar testes de caminhos e investigar se os diversos caminhos independentes são de fato alcançáveis.

Por fim, testes podem ser realizados em diversos ambientes. Testes unitários podem ser realizados no ambiente de desenvolvimento de um programador, enquanto testes de validação podem ser executados no ambiente de implantação do software. Desta forma, é possível identificar de forma proativa eventuais problemas de configuração e outras limitações dos ambientes de produção, para que o software apresente menos defeitos quando entrar em operação.

Atividade Extra

Obtenha algum código que já tenha desenvolvido em qualquer linguagem de programação. Realize um teste caixa preta e um teste caixa

branca para praticar. Lembre-se: se a descrição do software não apresentar informações suficientes para definir algum teste, então é sinal de que o código necessita de melhor descrição para ser considerado profissional.

Referência Bibliográfica

SOMMERVILLE, I. **Engenharia de Software**, 10ª edição. Addison Wesley, 2019.

Ir para exercício